

symbols (the option `-g`) and generate its assembly code with `objdump` as follows:

```
$ gcc -g test.c -o test
$ objdump -S ./test > asm
```

The `objdump` output is huge, so let's redirect the output to an `asm` file, which you can open in your favourite editor, and examine code from it. For example:

```
global_int1 = 20;
80483fa: c7 05 78 97 04 08 14 movl $0x14,0x8049778
```

I'm running CentOS on Intel hardware, so `asm` contains x86 assembly code. The `'global_int1'` variable is assigned the value `'20'` (0x14), which is shown in assembly in the next line. The address `'0x8049778'` is the location of `'global_int1'`; the `'movl'` instruction moves the data `0x14` (20) to this location. Next, look at the following:

```
From main()
return 0;
804848b: b8 00 00 00 00 mov $0x0,%eax
```

The return value is placed in the `'eax'` register. In this way, you can easily understand other instructions too, and enhance your assembly programming skills.

Strings

This tool prints the printable characters from a file and can help while debugging a C program. Also, if you get an executable file from the Internet or other source, which contains malicious code, you can get an idea of what it does by looking at printable characters. The tool `'strings'` is the solution. Here, run it on your own `test` executable:

```
$ strings ./test
/lib/ld-linux.so.2
q?Hm
__gmon_start__
libc.so.6
__IO_stdin_used
printf
memcpy
__libc_start_main
GLIBC_2.0
PTRH
QVhw
[^_]
[test1] global_int2 = %d
Hello
[test2] global_int1 = %d
[test3] global_string = %s
```

Strip

Another important tool is `'strip'`. While working on embedded systems projects, you must use memory (both RAM and storage) carefully. A few extra kilobytes can create problems. When you develop code, you need the symbols in it for debugging—but when you deploy it, there is no sense leaving the symbols in the executable file and wasting precious kilobytes of memory. So `strip` these symbols before deploying. Let's compile `'test.c'` again (`gcc test.c -o test`) and check:

```
$ file test
test: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV),
dynamically linked (uses shared libs), for GNU/Linux 2.6.15, not
stripped

$ ls -lh test
-rwxr-xr-x 1 manoj manoj 7.2K 2011-09-14 00:44 test
```

It's very clear that the 7.2K-size executable is not stripped. If you run `'nm'` on it, you will get the symbol details. Now `strip` this file to remove all symbols, with `strip` test, and check:

```
ls -lh test
-rwxr-xr-x 1 manoj manoj 5.4K 2011-09-14 00:47 test

$ file test
test: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV),
dynamically linked (uses shared libs), for GNU/Linux 2.6.15,
stripped

$ nm test
nm: test: no symbols
```

From the output, it's clear that the size is reduced to 5.4K after it is stripped, and `'nm'` has failed to find any symbols in the file. This is a better file to deploy to embedded systems.

Size

Finally, one more command that I need to cover is `'size'`, to get the sizes of different memory sections of an ELF file (executable):

```
$ size a.out
      text      data      bss      dec      hex filename
    1192       272        24      1488      5d0 a.out
```

In this output, `text`, `data` and `bss` should be familiar; `dec` is the total size of all sections (`text + data + bss`). 

By: Manoj Kumar

The author is a freelance developer and trainer. He leads a team in Linux kernel programming, Linux administration, cluster computing, embedded systems and QT/GTK programming on Linux. You can reach him at manoj@nixitsolutions.com and read his blog, www.open4india.blogspot.com, to share your Linux knowledge.